Unsupervised Learning Models

Unsupervised learning is used when the data does not have a target/output label.
The model tries to find hidden patterns, structure, similarity, or grouping in the data by itself.

Example:
You have customer data with age, salary, and spending score, but no label like “good customer” or “bad customer.”
An unsupervised model can group similar customers together.

Cluster = collection of data points that are more similar to each other than to points in other groups.

Clustering Techniques

Clustering is an unsupervised learning technique that groups data points based on:

- **Similarity** → points that are close or behave similarly are put in the same cluster
- **Dissimilarity** → points that are very different are placed in different clusters

So clustering helps find:

- natural groups
- hidden structure
- patterns in data

Example:
In an insurance dataset, customers with similar claim behavior may form one cluster, while high-risk customers may form another cluster.

Hierarchical Clustering

Hierarchical clustering builds clusters in a **tree-like structure**, called a **dendrogram**.

1) Agglomerative Clustering

- This is the bottom-up approach.
- each data point as its own cluster
- the algorithm keeps merging the most similar clusters
- this continues until only one cluster is left

Steps:

1. Start with every point as a separate cluster
2. Measure distance between clusters
3. Merge the two nearest clusters
4. Repeat until one big cluster remains

2) Divisive Clustering

This is the top-down approach.
It starts with:

- All data points in one single cluster
- The algorithm keeps **splitting clusters into smaller clusters**
- This continues until each point becomes its own cluster, or until the desired number of clusters is reached

Steps:

1. Start with all points in one cluster
2. Split the cluster into smaller clusters
3. Keep splitting further
4. Continue until stopping condition is reached

K-Means Clustering:

K-Means is an unsupervised learning algorithm used to group data into K clusters based on similarity.

Core Idea

- Choose **K (number of clusters)**
- Assign data points to nearest cluster
- Update cluster centers (centroids)
- Repeat until stable

Step 1: Initialize Centroids

- Randomly pick **K points** as initial **centroids (cluster centers)**

Step 2: Calculate Distance (Euclidean Distance)

- For each data point, calculate distance from each centroid.

Step 3: Assign Clusters

- Each point is assigned to the **nearest centroid**
- This forms **K clusters**

Step 4: Recalculate Centroids

- For each cluster, compute the **mean of all points**
- That mean becomes the **new centroid**

➢ Calculate Euclidean distance between data points and centroids.
➢ Initialize K centroids and compute distance of each data point to all centroid.
➢ Assign each point to the nearest centroid (cluster formation)
➢ Update centroids (mean of cluster), recompute distances, and repeat until convergence.

Elbow Method (K-Means)

The Elbow Method is used to determine the optimal number of clusters (K) in K-Means.
WCSS (Within-Cluster Sum of Squares) Vs K-Values

Core Idea

- As you increase **K (number of centroids)**:
  - Clusters become tighter
  - Error decreases
- But after a point:
  - Improvement becomes very small
  - That "bend" point is called the **Elbow**

It calculates the squared distance of each point from its cluster centroid.

Hierarchical Clustering

Hierarchical clustering builds a **tree-like structure of clusters**, called a **dendrogram**, to represent how data points are grouped at different levels.

Dendrogram:

A dendrogram is a tree diagram that shows:

- how clusters are formed (merging or splitting)
- the distance (similarity/dissimilarity) at which clusters are combined

dendrogram: it is agglomerative
Dendrogram is a visualization tool used in hierarchical clustering

- It is not the algorithm itself
- It is the output/representation of clustering

How Dendrogram is Built (Agglomerative)

1. Start with each data point as a separate cluster.
2. Merge closest clusters step-by-step.
3. Record the distance at each merge.
4. Build a tree structure (dendrogram).

Each vertical line's height = distance (or dissimilarity) at which two clusters are merged
Next merge always happens at equal or higher distance than previous merge.
It only increases upward.

Linkage Method:
A linkage method defines how the distance between two clusters is calculated.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)
DBSCAN is an unsupervised clustering algorithm that groups data based on density.

- Find arbitrary shaped clusters
- Detect outliers (noise)

DBSCAN is a density-based clustering algorithm that groups data points into clusters based on dense regions defined by epsilon (radius) and minimum points, while identifying sparse points as noise.
Epsilon:
maximum distance (radius) to search for neighbors.
Min Points:
Minimum number of points required to form a dense region.

Types of Points
1.Core Point
Number of points within Epsilon >=MinPts

2. Border Point
A point that is within Epsilon of a core point
But does NOT have enough neighbors to be a core point itself.

3) Noise Point (Outlier)
A point that is:

- Not a core point
- Not reachable from any core point
- Treated as **outlier**

K-Nearest Neighbors (KNN)
KNN is a supervised learning algorithm used for:

- Classification (most common)

- Regression

It is a **lazy learner**:

- No model is built during training
- Entire dataset is stored
- Computation happens during prediction

Similar data points exist close to each other in feature space.

Step 1: Choose K

- K = number of nearest neighbors (e.g., 3, 5, 7)

Step 2: Compute Distance

- Euclidean Distance

Step 3: Find K Nearest Points

- Sort distances
- Pick K smallest

Step 4: Make Prediction

Classification:

- Majority voting

Regression:

- Average of K values

Real-World Use Cases:

1) **Recommendation Systems**
2) **Fraud Detection (Insurance / Banking)**
3) **Customer Segmentation**

Uses when:
Small datasets
Low-dimensional data

K-Nearest Neighbors (KNN) is a supervised, instance-based learning algorithm that predicts the class or value of a data point by analyzing the majority class or average of its K nearest neighbors using a distance metric. It works for both Classification and Regression.

When a new data point comes:

- It looks at the **K nearest data points** (neighbors)
- Assigns the output based on those neighbors

Step 1: Choose K (number of neighbors)

- K = 3 → look at 3 nearest points

Step 2: Calculate distance from new point to all points

Euclidean Distance

Step 3: Find K nearest neighbors

- Sort distances
- Pick smallest K values

Step 4: Make Prediction

For Classification:

- Use **majority voting**

For Regression:

- Take **average of K values**

Support Vector Machine (SVM)
SVM tries to find the best boundary (hyperplane) that separates classes with maximum margin.
SVM is a supervised learning algorithm used for:

- Classification (most common)
- Regression
- Find the optimal boundary (hyperplane) that separates classes with maximum margin.
- Handles non-linear data
- **SVM finds** optimal hyperplane with maximum margin
- **Uses** support vectors **(critical points)**

Margin Concept
SVM does not just separate classes—it finds the maximum margin.

- Margin is Hyperplane and closest data points
- Closest points are called Support Vectors.
- Suppoer Vectors are Critical points that define the boundary.